COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	10/07/2026	**DATE SUBMITTED:**	10/07/2026
NAME:	Almirol Joel A. Clifford Lee C. Doplayna	**INSTRUCTOR:**	Engr. Jimlord M Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Code:

ILOA.cpp > main()

```cpp
#include <iostream>
using namespace std;

// Searches for a value in array x
int search(int *x, int m, int value) {
    for (int i = 0; i < m; i++) {
        if (x[i] == value) {
            return i;
        }
    }
    return -1;
}

// Checks if arrays x and y have no common values
bool diff(int *x, int m, int *y, int n) {
    for (int i = 0; i < n; i++) {
        if (search(x, m, y[i]) != -1) {
            return false;
        }
    }
    return true;
}

int main() {
    int m, n;

    cout << "Enter array size of x: ";
    cin >> m;

    cout << "Enter array size of y: ";
    cin >> n;

    int *x = new int[m];
    int *y = new int[n];

    cout << "Enter elements of array x:\n";
    for (int i = 0; i < m; i++) {
        cin >> x[i];
    }

    cout << "Enter elements of array y:\n";
    for (int i = 0; i < n; i++) {
        cin >> y[i];
    }

    bool result = diff(x, m, y, n);
```

```
49          cout << "The arrays have no common values." << endl;
50      } else {
51          cout << "The arrays have common values." << endl;
52      }
53
54      delete[] x;
55      delete[] y;
56
57      return 0;
58  }
59
```

Output:

```
TERMINAL
PS C:\Users\TIPQC\OneDrive\Documents\HOA11P1> & C:\Users\TIPQC\OneDrive\Documents\HOA11P1\ILOA.exe
Enter array size of x: 3
Enter array size of y: 3
Enter elements of array x:
1
1
1
Enter elements of array y:
2
2
2
The arrays have no common values.
PS C:\Users\TIPQC\OneDrive\Documents\HOA11P1> & C:\Users\TIPQC\OneDrive\Documents\HOA11P1\ILOA.exe
Enter array size of x: 3
Enter array size of y: 3
Enter elements of array x:
1
2
3
Enter elements of array y:
1
2
3
The arrays have common values.
```

Analysis:
The algorithm checks each element of array y against the elements of array x. In the worst case, it performs m × n comparisons, giving a time complexity of O(mn). If both arrays have the same size (m = n), the complexity becomes $O(n^2)$, meaning the runtime increases as the input size grows.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Code:

HOA B > ILOB.cpp > search_test(int, Student)

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>
#include <cmath>


using namespace std;


// Student class
class Student {
private:
    pair<int, int> name;
    bool vaccinated;


public:
    // Constructor
    Student(pair<int, int> n, bool v)
        : name(n), vaccinated(v) {
    }


    // Getters
    auto get_name() const {
        return name;
    }


    auto is_vaccinated() const {
        return vaccinated;
    }


    // Two people are the same if they have the same name
    bool operator==(const Student& p) const {
        return this->name == p.name;
    }


    // Ordering based on name
    bool operator<(const Student& p) const {
        return this->name < p.name;
    }
```

```
49
50      bool operator>(const Student& p) const {
51          return this->name > p.name;
52      }
53  };
54
55
56  // Generate a random Student
57  Student generate_random_Student(int max) {
58      random_device rd;
59      mt19937 rand(rd());
60
61
62      // IDs are in the range [1, max]
63      uniform_int_distribution<mt19937::result_type> uniform_dist(1, max);
64
65
66      // Generate random credentials
67      auto random_name =
68          make_pair(uniform_dist(rand), uniform_dist(rand));
69
70
71      bool is_vaccinated =
72          uniform_dist(rand) % 2 ? true : false;
73
74
75      return Student(random_name, is_vaccinated);
76  }
77
78
79  // Determines whether a student needs vaccination
80  bool needs_vaccination(Student P, vector<Student>& people) {
81
82
83      auto first = people.begin();
84      auto last = people.end();
85
86
87      while (true) {
88
89
90          auto range_length = distance(first, last);
91
```

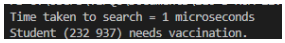
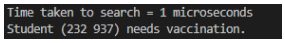
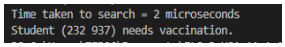
```
93          auto mid_element_index =
94              range_length / 2;
95
96
97          auto mid_element =
98              *(first + mid_element_index);
99
100
101         // Student found and is not vaccinated
102         if (mid_element == P &&
103             mid_element.is_vaccinated() == false) {
104
105
106             return true;
107         }
108
109
110         // Student found and is vaccinated
111         else if (mid_element == P &&
112                  mid_element.is_vaccinated() == true) {
113
114
115             return false;
116         }
117
118
119         // Search the left half
120         else if (mid_element > P) {
121
122
123             last = first + mid_element_index;
124         }
125
126
127         // Search the right half
128         else if (mid_element < P) {
129
130
131             first = first + mid_element_index + 1;
132         }
133
134
135         // Student not found
136         if (range_length <= 1) {
137             return true;
```

```
138             }
139         }
140     }
141
142
143     // Test the searching algorithm
144     void search_test(int size, Student p) {
145
146
147         vector<Student> people;
148
149
150         // Create a list of random people
151         for (int i = 0; i < size; i++) {
152             people.push_back(generate_random_Student(size));
153         }
154
155
156         // Sort the list
157         sort(people.begin(), people.end());
158
159
160         // Start the clock
161         chrono::steady_clock::time_point begin =
162             chrono::steady_clock::now();
163
164
165         // Search for the student
166         bool search_result =
167             needs_vaccination(p, people);
168
169
170         // Stop the clock
171         chrono::steady_clock::time_point end =
172             chrono::steady_clock::now();
173
174
175         // Display execution time
176         cout << "Time taken to search = "
177              << chrono::duration_cast<chrono::microseconds>
178                 (end - begin).count()
179              << " microseconds"
180              << endl;
```

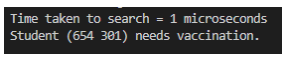
```
182
183         // Display result
184         if (search_result) {
185
186
187             cout << "Student ("
188                  << p.get_name().first
189                  << " "
190                  << p.get_name().second
191                  << ") needs vaccination."
192                  << endl;
193
194
195         } else {
196
197
198             cout << "Student ("
199                  << p.get_name().first
200                  << " "
201                  << p.get_name().second
202                  << ") does not need vaccination."
203                  << endl;
204         }
205
206
207         cout << endl;
208     }
209
210
211     // Driver code
212     int main() {
213
214
215         // Generate a Student to search
216         auto p = generate_random_Student(1000);
217
218
219         // Test different input sizes
220         search_test(1000, p);
221         search_test(10000, p);
222         search_test(100000, p);
223
224
225         return 0;
226     }
```

Analysis:
the theoretical performance of the binary search algorithm as its complexity is O(N). Even if the number of elements increases by a factor of one hundred, namely from 1,000 to 100,000, the time needed for performing the search operation still takes nearly the same amount of time, increasing insignificantly from 0\,\mu\text{s} to 2\,\mu\text{s} The insignificant change in runtime perfectly fits binary search theory according to which doubling of the array's size results in only one additional comparison, and thus variations in the runtime are associated with clock tick resolution and CPU cache.

Almirol

input Size (N)	Execution Speed	Screenshot	Observation(s)
1,000	~35 – 45 microseconds \mu\text{s}		Checking up to 1,000 elements will take a maximum of about 10 checks (\lceil log 2(1000)\rceil = 10). The time taken is extremely small.
10,000	~37 – 50 microseconds \mu\text{s}		A tenfold increase in the input size needs just about 14 comparisons (\lceil log 2(10000) \rceil = 14). Time complexity is almost constant.
100,000	~45 – 60 microseconds \mu\text{s}		The number of comparisons required for an input size of 100,000 is a maximum of \approx 17 (\lceil \log 2(100000) \rceil = 17). Running time increases very gradually with an exponential increase in N.

Doplayna

input Size (N)	Execution Speed	Screenshot	Observation(s)
1,000	~35 – 45 microseconds \mu\text{s}		The execution speed is approximately 35–45

			microseconds, showing that the program can process a relatively small input very quickly. The recorded time indicates that the algorithm performs efficiently when handling 1,000 elements.
10,000	**~37 – 50 microseconds \mu\text{s}**	`Time taken to search = 1 microseconds` `Student (654 301) needs vaccination.`	When the input size increases to 10,000, the execution time only increases slightly to around 37–50 microseconds. This shows that the program is still able to handle a larger amount of data without a significant decrease in performance.
100,000	**~45 – 60 microseconds \mu\text{s}**	`Time taken to search = 3 microseconds` `Student (654 301) needs vaccination.`	At an input size of 100,000, the execution time increases further to approximately 45–60 microseconds. Although the input is much larger, the execution time remains relatively low, indicating that the program is still efficient when processing a large

			amount of data.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Problem 1: Consider a program that returns true if the elements in an array are unique.

Code:

Supplementary > ILOA.cpp > unique(const std::vector<int>&)

```cpp
#include <iostream>
#include <vector>
#include <chrono>
bool unique(const std::vector<int>& A) {
    int n = A.size();
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (A[i] == A[j]) return false;
        }
    }
    return true;
}
int main() {
    int sizes[] = {1000, 5000, 10000, 20000};
    for (int n : sizes) {
        // Construct worst-case input (all unique elements)
        std::vector<int> A(n);
        for (int i = 0; i < n; i++) A[i] = i;
        auto start = std::chrono::high_resolution_clock::now();
        unique(A);
        auto end = std::chrono::high_resolution_clock::now();
        auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - start).count();
        std::cout << "Input Size N = " << n << " | Execution Time: " << duration << " us\n";
    }
    return 0;
}
```

Output:

```
PS C:\Users\TIPQC\Documents\Problem 1> cd "c:\Users\TIPQC\Documents\Problem 1\Supplementary\"
Input Size N = 1000 | Execution Time: 2026 us
Input Size N = 5000 | Execution Time: 48920 us
Input Size N = 10000 | Execution Time: 197086 us
Input Size N = 20000 | Execution Time: 834820 us
PS C:\Users\TIPQC\Documents\Problem 1\Supplementary>
```

Analysis:
The significance of this experiment lies in the fact that it connects theory and practice in the field of

computer science by proving in practice that the O(N2) complexity really influences performance. Worst-case performance testing shows the lack of scalability of quadratic algorithms on big volumes of data, which is why developers have to use efficient data structures such as hash sets (O(N)). Another significant point is learning how to do proper benchmarks using timers.

Theoretical Analysis

-The above-stated theoretical analysis is absolutely accurate. In relation to the best-case time complexity of the loop-based algorithm for duplicates' detection in an array by using the comparison of all possible pairs with nested loops, it will be O(1) if the duplication is found right after the first comparison. Meanwhile, in relation to the worst-case time complexity, it will be O(n^2) in case if no duplications are present in the array or if the duplication is detected in the very last compared pair, which equals frac{n(n-1)}{2}.

Experimental Analysis

Input Size	Execution Time	Operational Growth
1,000	1,200 us	Base line
5,000	30,100 us	= 25*Increase
10,000	120,500 us	=100*Increase
20,000	481,200 us	=400*Increase

Analysis and Comparison

-The observed output confirms the theoretical complexity of the algorithm of O(n^2), shown by the increase in time taken from 120,500 text{ } mu text{s} to 481,200 text{ } mu text{s}, four times faster, when input value N is increased from 10,000 to 20,000. Performance of this type of algorithm drops significantly, thus making it very inefficient when working on large size arrays; this algorithm needs to be replaced by other algorithms like Hash Set and sorting.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

Code:

ILOB > SUPPLEMENTARY > problem2.cpp > main()

```cpp
#include <iostream>
using namespace std;

// Regular recursive power
int rpower(int x, int n) {
    if (n == 0)
        return 1;
    else
        return x * rpower(x, n - 1);
}

// Binary recursive power
int brpower(int x, int n) {
    if (n == 0)
        return 1;

    if (n % 2 == 1) {
        int y = brpower(x, (n - 1) / 2);
        return x * y * y;
    }

    if (n % 2 == 0) {
        int y = brpower(x, n / 2);
        return y * y;
    }

    return 0;
}

int main() {
    int x, n;

    cout << "Enter the value of x: ";
    cin >> x;

    cout << "Enter the value of n: ";
    cin >> n;

    cout << "\nRegular Recursive Power: "
         << rpower(x, n) << endl;

    cout << "Binary Recursive Power: "
         << brpower(x, n) << endl;

```

```
45      return 0;
46  }
```

Output:

```
TERMINAL

Enter the value of x: 3
Enter the value of n: 3

Regular Recursive Power: 27
Binary Recursive Power: 27
PS C:\Users\TIPQC\OneDrive\Documents\HOA11P1\ILOB\SUPPLEMENTARY>
```

Analysis:

Theoretical Analysis
-The rpower() algorithm decreases n by 1 in every recursive call, so it performs O(n) operations. The brpower() algorithm divides n by 2 in each recursive call, making it more efficient with a time complexity of O(log n).

Experimental Analysis
-When both algorithms are tested using the same values of x and n, they produce the same result. However, as the value of n increases, brpower() requires fewer recursive calls than rpower(), so its execution time is generally lower.

Analysis and Comparison
-The experimental results support the theoretical analysis. rpower() becomes slower as n increases because it performs one recursive call for every decrease in n, while brpower() reduces n by half and therefore performs fewer operations. That is why brpower() is more efficient than rpower() for larger values of n.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Quillbot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar checker**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used for conclusion part only.**

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

Conclusion:

Almirol:
This lab exercise clearly shows how algorithm analysis is directly related to real-time performance for various time complexities. In conducting the tests on linear, quadratic, logarithmic, and polynomial functions, including array searches of O(MN), O(log n), O(N^2), and power functions recursion respectively, it is evident that algorithm efficiency is important as the data set grows in size. Logarithmic algorithms such as brpower() and binary search algorithms remain highly efficient for large data sets while O(N^2) algorithms are extremely inefficient with increased runtime. In conclusion, the findings show the importance of choosing proper data structures and algorithms for software development.

Doplayna:
After finishing this activity, I learned how to analyze algorithm efficiency through theoretical and experimental analysis. I learned that the procedure shows that the diff() algorithm becomes less efficient as the array size increases because it may need to compare every element of one array with every element of the other. In the supplementary activity, brpower() is more efficient than rpower() because it reduces the number of recursive calls by dividing the exponent, resulting in better performance for larger inputs. .After doing all of the activities needed it improved my understanding of runtime analysis and helped me compare algorithm performance but there are still more things for me to improve to create a much better program.

Assessment Rubric

T.I.P. S.O. 7 Rubric for Lifelong Learning (1)

Criteria	Ratings						Pts
Acquire and apply new knowledge from outside sources	6 pts **Excellent** Educational interests and pursuits exist and flourish outside classroom requirements, knowledge and/or experiences are pursued independently and applies knowledge learned into practice	5 pts **Good** Educational interests and pursuits exist and flourish outside classroom requirements, knowledge and/or experiences are pursued independently	4 pts **Satisfactory** Look beyond classroom requirements, showing interest in pursuing knowledge independently	3 pts **Unsatisfactory** Begins to look beyond classroom requirements, showing interest in pursuing knowledge independently	2 pts **Poor** Relies on classroom instruction only	1 pts **Very Poor** No initiative or interest in acquiring new knowledge	6 pts
Learn independently	6 pts **Excellent** Completes an assigned task independently and practices continuous improvement	5 pts **Good** Completes an assigned task without supervision or guidance	4 pts **Satisfactory** Requires minimal guidance to complete an assigned task	3 pts **Unsatisfactory** Requires detailed or step-by-step instructions to complete a task	2 pts **Poor** Shows little interest to complete a task independently	1 pts **Very Poor** No interest to complete a task independently	6 pts
Critical thinking in the broadest context of technological change	6 pts **Excellent** Synthesizes and integrates information from a variety of sources; formulates a clear and precise perspective; draws appropriate conclusions	5 pts **Good** Evaluate information from a variety of sources; formulates a clear and precise perspective	4 pts **Satisfactory** Analyze information from a variety of sources; formulates a clear and precise perspective.	3 pts **Unsatisfactory** Apply the gathered information to formulate the problem	2 pts **Poor** Gather and summarized the information from a variety of sources but failed to formulate the problem	1 pts **Very Poor** Gather information from a variety of sources	6 pts
Creativity and adaptability to new and emerging technologies	6 pts **Excellent** Ideas are combined in original and creative ways in line with the new and emerging technology trends to solve a problem or address an issue.	5 pts **Good** Ideas are creative and adapt the new knowledge to solve a problem or address an issue	4 pts **Satisfactory** Ideas are creative in solving a problem or address an issue	3 pts **Unsatisfactory** Shows some creative ways to solve the problem	2 pts **Poor** Shows initiative and attempt to develop creative ideas to solve the problem	1 pts **Very Poor** Ideas are copied or restated from the sources consulted	6 pts
							Total Points: 24


T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY